



Pontifícia Universidade Católica de Campinas

SISTEMAS DE INFORMAÇÃO

PADRÕES E ARQUITETURA DE SOFTWARE

Gabriella Cardoso Dos Santos RA: 23008318

Pedro Beresford Rocha RA: 23015967

CAMPINAS

2025

SUMÁRIO

SUMÁRIO	2
Introdução	5
1. Estudo Teórico dos Padrões	5
1.1 Singleton	5
Categoria	5
Propósito	5
Estrutura	5
Quando Utilizar	5
Iterações e Variações	6
Trade-offs	7
1.2 Repository	7
Categoria	7
Propósito	7
Estrutura	7
Quando Utilizar	8
Iterações e Variações	8
Trade-offs	8
1.3 Factory Method	9
Categoria	9
Propósito	9
Estrutura	9
Quando Utilizar	10
Iterações e Variações	10
Trade-offs	11
1.4 Strategy	11
Categoria	11
Propósito	11
Estrutura	11
Quando Utilizar	12
Iterações e Variações	12
Trade-offs	13
1.5 Observer	13
Categoria	13
Propósito	13
Estrutura	14
Quando Utilizar	14
Iterações e Variações	14
Trade-offs	15
1.6 Decorator	15

Categoria	15
Propósito	15
Estrutura	16
Quando Utilizar	16
Iterações e Variações	17
Trade-offs	17
2. Comparações Entre Padrões	18
2.1 Factory Method vs Builder	18
2.2 Strategy vs State	19
2.3 Observer vs Mediator	19
2.4 Decorator vs Proxy	20
3. Justificativas de Implementação	21
3.1 Singleton (ConfigManager e DatabaseManager)	21
Por que foi escolhido	21
Problema que resolve	21
Benefícios trazidos	22
Como seria sem o padrão	22
3.2 Repository (Acesso a Dados)	23
Por que foi escolhido	23
Problema que resolve	23
Benefícios trazidos	24
3.3 Factory Method (Criação de Pedidos e Produtos)	25
Por que foi escolhido	25
Problema que resolve	25
Benefícios trazidos	27
3.4 Strategy (Métodos de Pagamento)	28
Por que foi escolhido	28
Problema que resolve	28
Benefícios trazidos	30
3.5 Observer (Notificações de Pedidos)	31
Por que foi escolhido	31
Problema que resolve	31
Benefícios trazidos	33
3.6 Decorator (Extras de Pedidos)	34
Por que foi escolhido	34
Problema que resolve	34
Benefícios trazidos	36
4. Exemplos Práticos - Antes e Depois	36
Exemplo 1: Criação de Pedido Completo	36
ANTES (Sem padrões)	36
DEPOIS (Com todos os padrões)	38
5. Conclusão	40

Aprendizados Principais	40
Reflexões	40
Impacto no Projeto	41

Introdução

Este documento apresenta o estudo aprofundado de 6 padrões de projeto implementados em um sistema de e-commerce. O objetivo é demonstrar não apenas a implementação técnica, mas também o entendimento teórico, as motivações e os benefícios de cada padrão.

Os padrões foram escolhidos estrategicamente para resolver problemas reais encontrados em sistemas de e-commerce, desde a criação de objetos complexos até a comunicação entre componentes desacoplados.

1. Estudo Teórico dos Padrões

1.1 Singleton

Categoria

Padrão Criacional

Propósito

Garantir que uma classe tenha apenas uma única instância e fornecer um ponto global de acesso a essa instância.

Estrutura

```
class Singleton:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
```

Quando Utilizar

- Quando exatamente uma instância de uma classe deve existir
- Quando a instância deve ser acessível globalmente

- Quando a criação da instância é cara em termos de recursos
- Para gerenciar recursos compartilhados (conexões de banco, configurações)

Iterações e Variações

1. Singleton Clássico (Python)

```
class Singleton:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
```

2. Singleton Thread-Safe

```
import threading

class ThreadSafeSingleton:
    _instance = None
    _lock = threading.Lock()

    def __new__(cls):
        if cls._instance is None:
            with cls._lock:
                if cls._instance is None:
                    cls._instance = super().__new__(cls)
        return cls._instance
```

3. Singleton com Metaclass

```
class SingletonMeta(type):
    _instances = {}

    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            cls._instances[cls] = super().__call__(*args, **kwargs)
```

```
return cls._instances[cls]
```

Trade-offs

Vantagens:

- Controle estrito sobre única instância
- Economia de memória e recursos
- Acesso global conveniente
- Inicialização lazy (sob demanda)

Desvantagens:

- Pode violar o princípio de responsabilidade única
- Dificulta testes unitários (estado global)
- Pode criar dependências ocultas
- Problemas com concorrência se não thread-safe

1.2 Repository

Categoria

Padrão Arquitetural / Estrutural

Propósito

Encapsular a lógica de acesso a dados, fornecendo uma interface de coleção para acessar objetos de domínio, abstraindo os detalhes de persistência.

Estrutura

```
class Repository:
    def __init__(self, session):
        self.session = session

    def get_by_id(self, id): ...
    def get_all(self): ...
```

```
def create(self, obj): ...  
def update(self, obj): ...  
def delete(self, id): ...
```

Quando Utilizar

- Quando há necessidade de centralizar lógica de acesso a dados
- Ao trabalhar com ORMs ou múltiplas fontes de dados
- Para facilitar testes com mocks/stubs
- Quando se deseja desacoplar domínio de infraestrutura
- Para aplicar DDD (Domain-Driven Design)

Iterações e Variações

1. Repository Genérico

```
class GenericRepository[T]:  
    def __init__(self, model_class, session):  
        self.model_class = model_class  
        self.session = session
```

2. Repository Específico

```
class ProductRepository(BaseRepository):  
    def get_by_category(self, category):  
        # Método específico de produto
```

3. Repository com Unit of Work

```
class UnitOfWork:  
    def __init__(self):  
        self.products = ProductRepository()  
        self.orders = OrderRepository()  
  
    def commit(self):  
        self.session.commit()
```

Trade-offs

Vantagens:

- Desacoplamento entre lógica de negócio e acesso a dados
- Facilita testes (mock repositories)
- Centraliza queries e lógica de persistência
- Permite trocar fonte de dados facilmente
- Melhora manutenibilidade

Desvantagens:

- Camada adicional de abstração
- Pode ser over-engineering para apps simples
- Curva de aprendizado inicial
- Performance overhead se mal implementado

1.3 Factory Method

Categoria

Padrão Criacional

Propósito

Definir uma interface para criar um objeto, mas deixar as subclasses decidirem qual classe instanciar. O Factory Method permite que uma classe adie a instanciiação para subclasses.

Estrutura

```
# Creator abstrato
class Creator:
    def factory_method(self): pass

    def some_operation(self):
        product = self.factory_method()
        return product.operation()
```

```
# Creators concretos
class ConcreteCreatorA(Creator):
    def factory_method(self):
        return ConcreteProductA()
```

Quando Utilizar

- Quando não se sabe de antemão os tipos exatos de objetos
- Quando se quer fornecer uma biblioteca de componentes
- Para delegar a lógica de criação para subclasses
- Quando há famílias de objetos relacionados
- Para evitar acoplamento direto com classes concretas

Iterações e Variações

1. Factory Method Clássico

```
class OrderFactory:
    @classmethod
    def create_order(cls, order_type):
        creators = {
            'regular': RegularOrderCreator(),
            'express': ExpressOrderCreator()
        }
        return creators[order_type].create()
```

2. Simple Factory (Simplificação)

```
class OrderFactory:
    @staticmethod
    def create_order(order_type):
        if order_type == 'regular':
            return RegularOrder()
        elif order_type == 'express':
            return ExpressOrder()
```

3. Abstract Factory (Evolução)

```
class AbstractFactory:
    def create_order(self): pass
    def create_payment(self): pass
```

Trade-offs

Vantagens:

- Evita acoplamento entre criador e produtos concretos
- Princípio Open/Closed (aberto para extensão, fechado para modificação)
- Centraliza lógica de criação
- Facilita adicionar novos tipos
- Código mais testável

Desvantagens:

- Pode complicar o código com muitas subclasses
- Overhead de classes adicionais
- Complexidade inicial maior

1.4 Strategy

Categoria

Padrão Comportamental

Propósito

Definir uma família de algoritmos, encapsular cada um deles e torná-los intercambiáveis. O Strategy permite que o algoritmo varie independentemente dos clientes que o utilizam.

Estrutura

```
# Interface Strategy
class Strategy:
    def execute(self, data): pass
```

```
# Estratégias concretas
class ConcreteStrategyA(Strategy):
    def execute(self, data):
        return "Estratégia A"

# Contexto
class Context:
    def __init__(self, strategy):
        self.strategy = strategy

    def do_something(self, data):
        return self.strategy.execute(data)
```

Quando Utilizar

- Quando há múltiplas variantes de um algoritmo
- Para eliminar condicionais complexas (if/elif)
- Quando diferentes comportamentos são necessários em runtime
- Para isolar detalhes de implementação de algoritmos
- Ao aplicar o princípio Open/Closed

Iterações e Variações

1. Strategy com Interface Explícita

```
class PaymentStrategy(ABC):
    @abstractmethod
    def process_payment(self, amount): pass
```

2. Strategy com Funções (Python idiomático)

```
def credit_card_payment(amount):
    ...

def pix_payment(amount):
```

```
...

strategies = {
    'credit': credit_card_payment,
    'pix': pix_payment
}
```

3. Strategy com Dependency Injection

```
class OrderService:
    def __init__(self, payment_strategy: PaymentStrategy):
        self.payment_strategy = payment_strategy
```

Trade-offs

Vantagens:

- Elimina condicionais complexas
- Facilita adicionar novas estratégias
- Cada estratégia é testável independentemente
- Princípio Open/Closed
- Código mais limpo e manutenível

Desvantagens:

- Aumenta número de classes/objetos
- Cliente precisa conhecer diferenças entre estratégias
- Overhead de comunicação entre objetos

1.5 Observer

Categoria

Padrão Comportamental

Propósito

Definir uma dependência um-para-muitos entre objetos, de modo que quando um objeto muda de estado, todos os seus dependentes são notificados e atualizados automaticamente.

Estrutura

```
# Subject
class Subject:
    def __init__(self):
        self._observers = []

    def attach(self, observer):
        self._observers.append(observer)

    def notify(self, data):
        for observer in self._observers:
            observer.update(data)

# Observer
class Observer(ABC):
    @abstractmethod
    def update(self, data): pass
```

Quando Utilizar

- Quando mudança em um objeto requer mudanças em outros
- Quando não se sabe quantos objetos precisam ser notificados
- Para implementar event-driven architecture
- Quando se quer baixo acoplamento entre objetos
- Para implementar sistemas de notificação

Iterações e Variações

1. Push Model (Subject envia dados)

```
def notify(self, data):
    for observer in self._observers:
```

```
observer.update(data) # Envia dados
```

2. Pull Model (Observer busca dados)

```
def notify(self):  
    for observer in self._observers:  
        observer.update(self) # Observer busca do subject
```

3. Event Bus (Pub/Sub)

```
class EventBus:  
    def subscribe(self, event_type, handler): ...  
    def publish(self, event_type, data): ...
```

Trade-offs

Vantagens:

- Baixo acoplamento entre subject e observers
- Suporta broadcast de comunicação
- Fácil adicionar/remover observers dinamicamente
- Princípio Open/Closed
- Permite reação a eventos em tempo real

Desvantagens:

- Observers são notificados em ordem aleatória
- Pode causar cascata de atualizações
- Difícil rastrear fluxo de execução
- Memory leaks se observers não forem removidos

1.6 Decorator

Categoria

Padrão Estrutural

Propósito

Anexar responsabilidades adicionais a um objeto dinamicamente. Os Decorators fornecem uma alternativa flexível ao uso de subclasses para estender funcionalidades.

Estrutura

```
# Component interface
class Component(ABC):
    @abstractmethod
    def operation(self): pass

# Concrete component
class ConcreteComponent(Component):
    def operation(self):
        return "Base"

# Decorator
class Decorator(Component):
    def __init__(self, component):
        self._component = component

    def operation(self):
        return self._component.operation()

# Concrete decorator
class ConcreteDecorator(Decorator):
    def operation(self):
        return f"Decorated({self._component.operation()})"
```

Quando Utilizar

- Para adicionar responsabilidades a objetos em runtime
- Quando herança não é prática
- Para evitar explosão de subclasses
- Quando se quer combinar comportamentos

- Para seguir o princípio Open/Closed

Iterações e Variações

1. Decorator Clássico (GoF)

```
class Component:
    def operation(self): pass

class Decorator(Component):
    def __init__(self, component):
        self.component = component
```

2. Decorator com Python @decorator

```
def log_decorator(func):
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        return func(*args, **kwargs)
    return wrapper
```

3. Chaining Decorators

```
order = BaseOrder(100)
order = GiftWrapDecorator(order)
order = InsuranceDecorator(order)
```

Trade-offs

Vantagens:

- Mais flexível que herança
- Adiciona responsabilidades em runtime
- Evita classes com muitas funcionalidades
- Permite combinação de decorators
- Princípio de Responsabilidade Única

Desvantagens:

- Pode resultar em muitos objetos pequenos

- Difícil de debugar com muitos decorators
- Ordem dos decorators pode importar
- Complexidade de identidade de objetos

2. Comparações Entre Padrões

2.1 Factory Method vs Builder

Semelhanças:

- Ambos são padrões criacionais
- Ambos abstraem a criação de objetos
- Ambos promovem baixo acoplamento

Diferenças:

- **Factory Method**: Cria objetos de diferentes tipos/classes
- **Builder**: Constrói objetos complexos passo a passo (mesma classe)

Quando usar cada um:

- **Factory Method**: Quando há famílias de produtos relacionados
- **Builder**: Quando objeto tem muitos parâmetros opcionais

Exemplo:

```
# Factory Method
order = OrderFactory.create_order('express', data) # Cria
ExpressOrder

# Builder
order = OrderBuilder() \
.with_customer(customer) \
.with_items(items) \
.with_payment('pix') \
```

```
.build() # Constrói um Order passo a passo
```

2.2 Strategy vs State

Semelhanças:

- Ambos encapsulam comportamento variável
- Ambos usam composição ao invés de herança
- Estrutura similar (contexto + estratégias/estados)

Diferenças:

- **Strategy**: Cliente escolhe estratégia explicitamente
- **State**: Objeto muda estado automaticamente
- **Strategy**: Estratégias são independentes
- **State**: Estados conhecem transições entre si

Quando usar cada um:

- **Strategy**: Quando comportamento é configurável
- **State**: Quando comportamento depende do estado interno

Exemplo:

```
# Strategy - Cliente escolhe
payment_service = PaymentService(PixPayment()) # Escolha
explicita

# State - Objeto transiciona
order.status = 'pending' # Estado inicial
order.confirm() # Muda para 'confirmed' automaticamente
```

2.3 Observer vs Mediator

Semelhanças:

- Ambos desacoplam objetos
- Ambos lidam com comunicação entre objetos
- Ambos são comportamentais

Diferenças:

- **Observer**: Comunicação um-para-muitos (broadcast)
- **Mediator**: Comunicação muitos-para-muitos (centralizada)
- **Observer**: Subject não conhece observers
- **Mediator**: Componentes conhecem o mediator

Quando usar cada um:

- **Observer**: Para notificações/eventos
- **Mediator**: Para coordenar interações complexas

Exemplo:

```
# Observer
order_subject.notify(data) # Notifica TODOS observers

# Mediator
mediator.coordinate(component_a, component_b) # Coordena
interação
```

2.4 Decorator vs Proxy

Semelhanças:

- Ambos são estruturais
- Ambos usam composição
- Ambos encapsulam objeto real

Diferenças:

- **Decorator**: Adiciona comportamento
- **Proxy**: Controla acesso
- **Decorator**: Pode empilhar múltiplos
- **Proxy**: Geralmente único

Quando usar cada um:

- ****Decorator****: Para estender funcionalidade
- ****Proxy****: Para lazy loading, cache, controle de acesso

Exemplo:

```
# Decorator - Adiciona funcionalidade
order = GiftWrapDecorator(InsuranceDecorator(base_order))

# Proxy - Controla acesso
real_order = OrderProxy(expensive_order) # Lazy loading
```

3. Justificativas de Implementação

3.1 Singleton (ConfigManager e DatabaseManager)

Por que foi escolhido

Em qualquer aplicação, configurações e conexões de banco de dados são recursos que devem existir em instância única. Múltiplas instâncias causariam:

- Inconsistência de configurações
- Desperdício de conexões de banco
- Uso excessivo de memória
- Problemas de sincronização

Problema que resolve

Sem o padrão:

```
# Cada módulo cria sua própria instância
config1 = Settings() # Instância 1
config2 = Settings() # Instância 2 (diferente!)

# Problemas:
- Configurações podem divergir
- Alterações em uma não afetam a outra
- Uso desnecessário de memória
```

Com o padrão:

```
# Sempre a mesma instância
config1 = get_config() # Instância única
config2 = get_config() # Mesma instância!

assert config1 is config2 # True
```

Benefícios trazidos

Manutenibilidade:

- Configurações centralizadas
- Fácil modificar comportamento global
- Ponto único de acesso

Escalabilidade:

- Pool de conexões gerenciado
- Uso eficiente de recursos
- Melhor performance

Arquitetura:

- Separação de preocupações
- Estado global controlado
- Facilita dependency injection

Como seria sem o padrão

```
# CÓDIGO RUIM - Sem Singleton

class OrderService:
    def __init__(self):
        # Cada service cria sua própria conexão!
        self.engine = create_engine('sqlite:///./db.sqlite')
        self.session = Session(self.engine)
```

```

class ProductService:
    def __init__(self):
        # Outra conexão diferente!
        self.engine = create_engine('sqlite:///./db.sqlite')
        self.session = Session(self.engine)

    # Problemas:
    # - Múltiplas conexões desnecessárias
    # - Desperdício de recursos
    # - Difícil gerenciar transações

```

3.2 Repository (Acesso a Dados)

Por que foi escolhido

Acesso a dados é uma preocupação transversal que aparece em todo o sistema. Sem Repository, queries SQLAlchemy estariam espalhadas por services, controllers e até views, causando:

- Alto acoplamento com tecnologia de persistência
- Duplicação de queries
- Dificuldade em testes
- Violação do princípio DRY

Problema que resolve

Sem o padrão:

```

# CÓDIGO RUIM - Queries espalhadas nos services

class OrderService:
    def get_order(self, order_id):
        # Query SQLAlchemy diretamente no service
        return session.query(Order).filter(Order.id ==
            order_id).first()

    def get_customer_orders(self, customer_id):

```

```
# Outra query SQL no service
return session.query(Order).filter(
    Order.customer_id == customer_id
).order_by(Order.created_at.desc()).all()

# Problemas:
# - Service conhece detalhes do SQLAlchemy
# - Difícil mockar para testes
# - Queries duplicadas
# - Se trocar de banco, muda TUDO
```

Com o padrão:

```
# CÓDIGO BOM - Repository abstrai acesso

class OrderRepository:
    def get_by_id(self, order_id):
        return self.session.query(Order).filter(Order.id ==
            order_id).first()

    def get_by_customer(self, customer_id):
        return self.session.query(Order).filter(
            Order.customer_id == customer_id
        ).order_by(Order.created_at.desc()).all()

class OrderService:
    def __init__(self, order_repo: OrderRepository):
        self.order_repo = order_repo

    def get_order(self, order_id):
        return self.order_repo.get_by_id(order_id) # Simples e
            testável!
```

Benefícios trazidos

Manutenibilidade:

- Queries centralizadas

- Fácil modificar lógica de acesso
- Código DRY

Escalabilidade:

- Fácil adicionar cache
- Fácil trocar fonte de dados
- Queries otimizadas em um lugar

Testabilidade:

- Fácil mockar repositories
- Testes unitários desacoplados
- Testes de integração focados

3.3 Factory Method (Criação de Pedidos e Produtos)

Por que foi escolhido

E-commerce tem diferentes tipos de pedidos (regular, expresso, internacional) e produtos (físico, digital, assinatura). Cada tipo tem regras específicas de criação, mas compartilha interface comum. Sem Factory Method, teríamos condicionais complexas espalhadas pelo código.

Problema que resolve

Sem o padrão:

```
# CÓDIGO RUIM - Condicionais complexas

class OrderService:
    def create_order(self, order_type, data):
        if order_type == 'regular':
            order = Order(
                type='regular',
                subtotal=data['subtotal'],
                total=data['subtotal'] # Sem taxa
```

```

)
elif order_type == 'express':
    express_fee = data['subtotal'] * 0.15
    order = Order(
        type='express',
        subtotal=data['subtotal'],
        extras_cost=express_fee,
        total=data['subtotal'] + express_fee
    )
elif order_type == 'international':
    intl_fee = data['subtotal'] * 0.30
    order = Order(
        type='international',
        subtotal=data['subtotal'],
        extras_cost=intl_fee,
        total=data['subtotal'] + intl_fee
    )
else:
    raise ValueError("Tipo inválido")

return order

# Problemas:
# - Método gigante com muitos ifs
# - Difícil adicionar novo tipo (modifica código existente)
# - Lógica de criação espalhada
# - Viola Open/Closed Principle

```

Com o padrão:

```

# CÓDIGO BOM - Factory Method

class OrderFactory:
    _creators = {
        'regular': RegularOrderCreator(),
        'express': ExpressOrderCreator(),
    }

```

```
'international': InternationalOrderCreator(),
}

@classmethod
def create_order(cls, order_type, order_data):
    creator = cls._creators.get(order_type)
    if not creator:
        raise ValueError(f"Tipo desconhecido: {order_type}")
    return creator.create_order(order_data)

# Benefícios:
# - Sem condicionais
# - Cada creator tem sua lógica
# - Adicionar novo tipo: criar nova classe (não modifica código existente!)
# - Open/Closed Principle
```

Benefícios trazidos

Manutenibilidade:

- Lógica de criação encapsulada
- Cada tipo em sua própria classe
- Fácil entender e modificar

Escalabilidade:

- Adicionar novos tipos sem alterar código existente
- Cada creator é independente
- Fácil testar isoladamente

Arquitetura:

- Princípio Open/Closed
- Single Responsibility
- Código limpo e organizado

3.4 Strategy (Métodos de Pagamento)

Por que foi escolhido

E-commerce precisa suportar múltiplos métodos de pagamento (cartão de crédito, PIX, boleto), cada um com:

- Validações específicas
- Processamento diferente
- Retornos distintos

Sem Strategy, teríamos um método `process\_payment` gigante com if/elif para cada tipo.

Problema que resolve

Sem o padrão:

```
# CÓDIGO RUIM - Condicionais gigantes

class PaymentService:
    def process_payment(self, method, amount, details):
        if method == 'credit_card':
            # Validação de cartão
            if not self._validate_card(details):
                return {'success': False}
            # Processamento de cartão
            transaction_id = self._process_card(amount, details)
            return {'success': True, 'transaction_id': transaction_id}

        elif method == 'pix':
            # Validação PIX
            if amount <= 0:
                return {'success': False}
            # Gera código PIX
            pix_code = self._generate_pix_code(amount)
            return {'success': True, 'pix_code': pix_code}
```

```

elif method == 'boleto':
    # Validação boleto
    if 'payer_cpf' not in details:
        return {'success': False}
    # Gera boleto
    boleto_code = self._generate_boleto(amount, details)
    return {'success': True, 'boleto_code': boleto_code}

else:
    return {'success': False, 'error': 'Método inválido'}

# Problemas:
# - Método gigante (violates SRP)
# - Difícil testar cada método isoladamente
# - Adicionar novo método = modificar este método
# - Lógicas misturadas

```

Com o padrão:

```

# CÓDIGO BOM - Strategy Pattern

class CreditCardPayment(PaymentStrategy):
    def process_payment(self, amount, details):
        if not self._validate_card(details):
            return {'success': False}
        transaction_id = self._process_card(amount, details)
        return {'success': True, 'transaction_id': transaction_id}

class PixPayment(PaymentStrategy):
    def process_payment(self, amount, details):
        if amount <= 0:
            return {'success': False}
        pix_code = self._generate_pix_code(amount)
        return {'success': True, 'pix_code': pix_code}

```

```
class PaymentService:
    def __init__(self):
        self._strategies = {
            'credit_card': CreditCardPayment(),
            'pix': PixPayment(),
            'boleto': BoletoPayment()
        }

    def process(self, method, amount, details):
        strategy = self._strategies.get(method)
        return strategy.process_payment(amount, details)

# Benefícios:
# - Cada estratégia é independente
# - Fácil testar cada uma
# - Adicionar novo método: criar nova classe
# - Código limpo e organizado
```

Benefícios trazidos

Manutenibilidade:

- Cada método em sua própria classe
- Lógica isolada e focada
- Fácil modificar um método sem afetar outros

Escalabilidade:

- Adicionar novos métodos de pagamento facilmente
- Cada estratégia testável isoladamente
- Permite diferentes implementações

Testabilidade:

- Testes unitários focados
- Mocks fáceis de criar

- Cobertura completa

3.5 Observer (Notificações de Pedidos)

Por que foi escolhido

Quando um pedido muda de status, múltiplos sistemas precisam ser notificados:

- Cliente precisa receber email
- Cliente precisa receber SMS
- Sistema precisa logar a mudança
- Dashboard precisa atualizar

Sem Observer, o código de mudança de status teria chamadas diretas para cada sistema, criando alto acoplamento.

Problema que resolve

Sem o padrão:

```
# CÓDIGO RUIM - Alto acoplamento

class OrderService:
    def update_status(self, order_id, new_status):
        order = self.order_repo.get_by_id(order_id)
        order.status = new_status
        self.order_repo.update(order)

        # Acoplamento direto com cada sistema
        self.email_service.send_email(
            order.customer.email,
            f"Seu pedido está {new_status}"
        )

        self.sms_service.send_sms(
            order.customer.phone,
```

```

f"Pedido {order_id}: {new_status}"
)

self.logger.log(
f"Order {order_id} changed to {new_status}"
)

self.dashboard.update_order(order_id, new_status)

# Se adicionar novo sistema, modifica AQUI!

# Problemas:
# - OrderService conhece TODOS os sistemas
# - Alto acoplamento
# - Difícil adicionar/remover sistemas
# - Difícil testar
# - Viola SRP

```

Com o padrão:

```

# CÓDIGO BOM - Observer Pattern

class OrderService:
    def __init__(self):
        self.notification_service = NotificationService() #
        Subject

    # Observers já estão registrados no NotificationService

    def update_status(self, order_id, new_status):
        order = self.order_repo.get_by_id(order_id)
        old_status = order.status
        order.status = new_status
        self.order_repo.update(order)

    # Apenas notifica - não conhece observers!
    self.notification_service.notify_order_status_change(
        order_id=order.id,

```



```
customer_name=order.customer.name,
customer_email=order.customer.email,
old_status=old_status,
new_status=new_status
)

# Notification Service (Subject)
class NotificationService:
    def __init__(self):
        self.subject = OrderSubject()
        self.subject.attach(EmailNotifier())
        self.subject.attach(SmsNotifier())
        self.subject.attach(LogNotifier())
        # Fácil adicionar/remover observers!

# Benefícios:
# - OrderService não conhece observers
# - Baixo acoplamento
# - Fácil adicionar novos observers
# - Fácil testar
```

Benefícios trazidos

Manutenibilidade:

- Baixo acoplamento entre componentes
- Fácil adicionar/remover notificadores
- Mudanças localizadas

Escalabilidade:

- Novos observers sem modificar código existente
- Observers podem ser adicionados dinamicamente
- Sistema reativo e extensível

Testabilidade:

- Fácil mockar observers
- Testes focados em cada notificador
- Teste de integração simplificado

3.6 Decorator (Extras de Pedidos)

Por que foi escolhido

E-commerce oferece extras opcionais para pedidos:

- Embalagem presente (+R\$ 10)
- Seguro padrão (+5%)
- Seguro premium (+8%)

Sem Decorator, precisaríamos de subclasses para cada combinação:

- `OrderWithGiftWrap`
- `OrderWithInsurance`
- `OrderWithGiftWrapAndInsurance`
- `OrderWithGiftWrapAndPremiumInsurance`
- ... (explosão de classes!)

Problema que resolve

Sem o padrão:

```
# CÓDIGO RUIM - Explosão de subclasses

class Order:
    def get_cost(self):
        return self.subtotal

class OrderWithGiftWrap(Order):
    def get_cost(self):
        return self.subtotal + 10.0
```

```

class OrderWithInsurance(Order):
    def get_cost(self):
        return self.subtotal * 1.05

class OrderWithGiftWrapAndInsurance(Order):
    def get_cost(self):
        return (self.subtotal + 10.0) * 1.05

class OrderWithGiftWrapAndPremiumInsurance(Order):
    def get_cost(self):
        return (self.subtotal + 10.0) * 1.08

# Problemas:
# - Explosão combinatória de classes
# - Código duplicado
# - Difícil adicionar novo extra
# - 3 extras = 8 classes (2^3)
# - 4 extras = 16 classes (2^4)

```

Com o padrão:

```

# CÓDIGO BOM - Decorator Pattern

# Base
order = BaseOrder(subtotal=100.0)

# Adiciona extras dinamicamente
if has_gift_wrap:
    order = GiftWrapDecorator(order)

if has_insurance:
    insurance_type = 'premium' if premium else 'standard'
    order = InsuranceDecorator(order, insurance_type)

cost = order.get_cost() # Calcula com todos os extras!

```

```
# Benefícios:
# - Apenas 1 classe base + N decorators
# - Fácil combinar em qualquer ordem
# - Adicionar novo extra: 1 nova classe decorator
# - Flexível e extensível
```

Benefícios trazidos

Manutenibilidade:

- Evita explosão de subclasses
- Cada decorator é independente
- Fácil entender e modificar

Escalabilidade:

- Adicionar novos extras facilmente
- Combinações infinitas
- Código DRY

Flexibilidade:

- Decorators aplicados em runtime
- Ordem configurável
- Reutilizável

4. Exemplos Práticos - Antes e Depois

Exemplo 1: Criação de Pedido Completo

ANTES (Sem padrões)

```
def create_order(customer_data, items, order_type,
payment_method, has_gift, has_insurance):
    # Cria/obtem cliente manualmente
    customer = session.query(Customer).filter(
        Customer.email == customer_data['email']
    ).first()
```

```
if not customer:
    customer = Customer(**customer_data)
    session.add(customer)
    session.flush()

# Calcula subtotal manualmente
subtotal = 0
for item in items:
    product = session.query(Product).get(item['product_id'])
    subtotal += product.price * item['quantity']

# Calcula extras com IFs
extras_cost = 0
if has_gift:
    extras_cost += 10.0
if has_insurance:
    extras_cost += subtotal * 0.05

# Cria pedido com IFs gigantes
if order_type == 'regular':
    total = subtotal + extras_cost
elif order_type == 'express':
    total = subtotal + extras_cost + (subtotal * 0.15)
elif order_type == 'international':
    total = subtotal + extras_cost + (subtotal * 0.30)

order = Order(
    customer_id=customer.id,
    type=order_type,
    subtotal=subtotal,
    total=total,
    status='pending'
)
session.add(order)
session.flush()
```

```

# Processa pagamento com IFs
if payment_method == 'credit_card':
# Código de cartão
pass
elif payment_method == 'pix':
# Código PIX
pass
elif payment_method == 'boleto':
# Código boleto
pass

# Notifica manualmente
send_email(customer.email, f"Pedido {order.id} criado")
send_sms(customer.phone, f"Pedido {order.id}")
log(f"Order {order.id} created")

session.commit()
return order

# PROBLEMAS:
# - Função gigante (100+ linhas)
# - Viola SRP
# - Difícil testar
# - Lógica duplicada
# - Alto acoplamento
# - Difícil manter

```

DEPOIS (Com todos os padrões)

```

class OrderService:
def create_order(self, order_data: OrderCreate) -> Order:
# 1. Repository: Obtém/cria cliente
customer =
self._get_or_create_customer(order_data.customer)

```

```
# 2. Repository: Calcula subtotal
items_data, subtotal =
self._calculate_order_items(order_data.items)

# 3. Decorator: Calcula extras
extras_cost = self._calculate_extras_cost(
    subtotal,
    order_data.extras.gift_wrap,
    order_data.extras.insurance
)

# 4. Factory Method: Cria pedido
order = OrderFactory.create_order(
    order_data.order_type,
    {
        'customer_id': customer.id,
        'payment_method': order_data.payment_method,
        'subtotal': subtotal,
        'extras_cost': extras_cost
    }
)

# 5. Strategy: Processa pagamento
payment_result = self._process_payment(
    order_data.payment_method,
    order.total,
    self._prepare_payment_details(order_data, customer)
)

# 6. Repository: Persiste
created_order = self.order_repo.create_with_items(order,
    items_data)

# 7. Observer: Notifica
self.notification_service.notify_order_status_change(...)
```

```
return created_order

# BENEFÍCIOS:
# - Método limpo e legível (30 linhas)
# - Cada padrão resolve um problema
# - Fácil testar (mocks)
# - Baixo acoplamento
# - Alta coesão
# - Fácil manter e estender
```

5. Conclusão

Aprendizados Principais

1. **Padrões Resolvem Problemas Reais:** Cada padrão implementado resolve um problema específico do sistema de e-commerce, não sendo aplicado de forma forçada.

2. **Integração Natural:** Os padrões trabalham juntos harmoniosamente. O `OrderService` demonstra como múltiplos padrões podem colaborar para resolver um problema complexo.

3. **Manutenibilidade:** Código com padrões é significativamente mais fácil de manter, entender e modificar.

4. **Extensibilidade:** Adicionar novos tipos de pedidos, métodos de pagamento ou notificadores é trivial com os padrões implementados.

5. **Testabilidade:** Cada componente pode ser testado isoladamente graças ao baixo acoplamento proporcionado pelos padrões.

Reflexões

Sobre Singleton:

- Útil para recursos únicos, mas deve ser usado com cuidado
- Facilita muito a gestão de configurações e conexões

- Thread-safety é importante em ambientes de produção

Sobre Repository:

- Essencial para desacoplar lógica de negócio de persistência
- Facilita testes unitários dramaticamente
- Permite trocar tecnologia de banco sem afetar lógica

Sobre Factory Method:

- Elimina condicionais complexas elegantemente
- Torna código muito mais extensível
- Exemplifica perfeitamente o princípio Open/Closed

Sobre Strategy:

- Perfeito para algoritmos intercambiáveis
- Cada estratégia é testável independentemente
- Código muito mais limpo sem if/elif gigantes

Sobre Observer:

- Essencial para sistemas reativos
- Baixo acoplamento entre componentes
- Fácil adicionar novas funcionalidades

Sobre Decorator:

- Alternativa elegante à herança
- Flexibilidade em runtime
- Evita explosão de classes

Impacto no Projeto

A aplicação dos padrões transformou um código que seria:

- Difícil de manter
- Difícil de testar

- Difícil de estender
- Alto acoplamento
- Código duplicado

Em um código que é:

- Fácil de manter
- Altamente testável
- Facilmente extensível
- Baixo acoplamento
- DRY (Don't Repeat Yourself)